

Topic 3

Intersystem communication

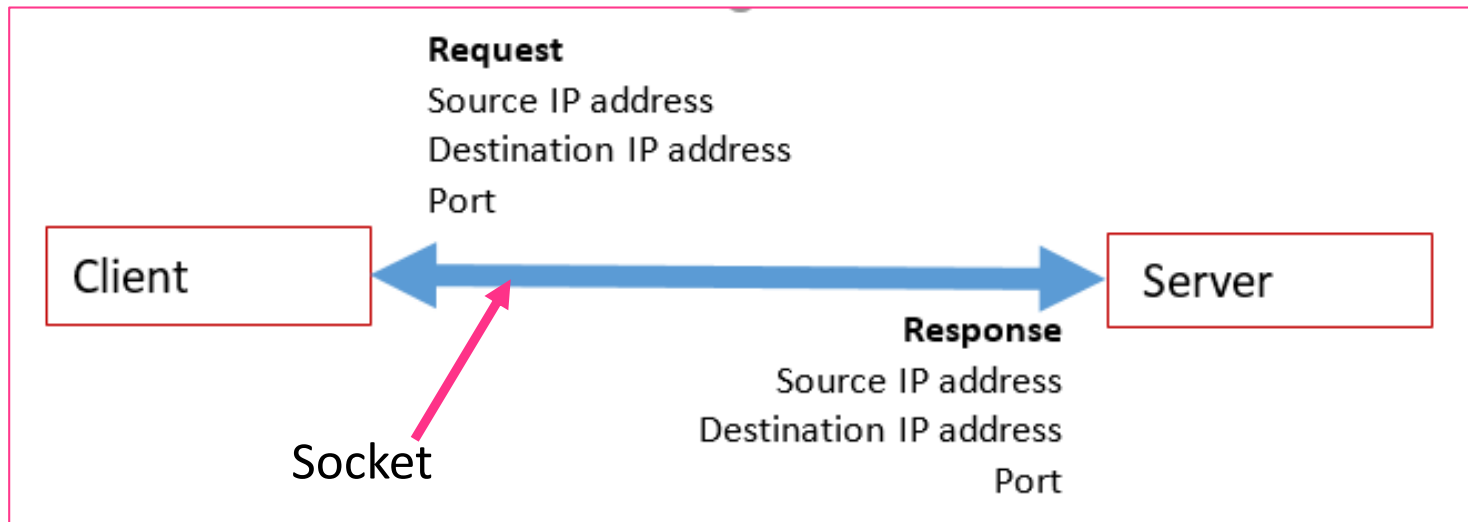
Part 2: Network programming

Agenda

- Overview of the previous lecture
- Introduction
- Distributed applications
 1. Features of distributed environment
 2. Distributed applications: advantages and challenges
 3. Distributed architecture: DOA, SOA
- ~~■ Network programming~~
 - ~~1. Sockets and streams~~
 - ~~2. Accessing URL~~
- ~~■ CORBA~~
- Remote Message Invocation
- ~~■ Web services~~
- ~~■ Messaging services~~
- Important resources
- Summary

❑ **Part 2: Network programming**

Network programming



- ❑ Network programming: writing programs which execute on 2 or more devices on a network
- ❑ The following are important for communication between client and server
 1. Sockets for low-level network connection
 2. Streams (i.e., data flow)
 3. Protocols (to control data flow)
 4. Other objects, e.g., exception handling; high-level connection to load resources from servers

Network programming

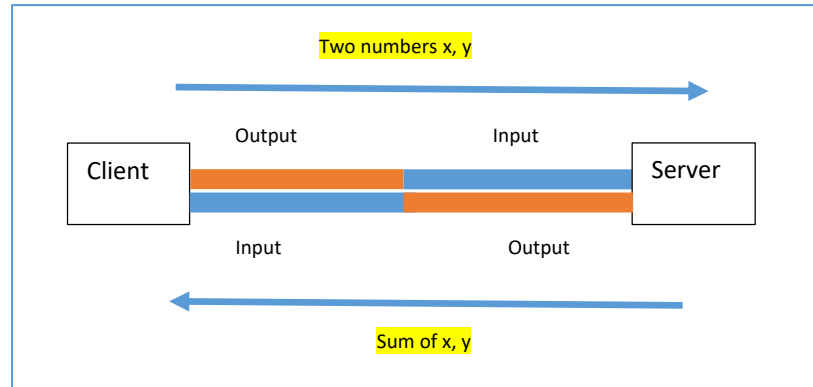
- ❑ Java defines two types of communication channels in the java.net package
 1. **Socket (for TCP/IP: Transport Control Protocol/Internet Protocol network)**
 2. **DatagramSocket (for UDP: User Datagram Protocol);**
 - Unreliable
- ❑ **We'll focus on TCP/IP**
- ❑ TCP/IP provides a reliable channel for a client and a server to communication
 1. A socket is an object which represents this connection
 2. Both the client and server can read from (or write to) the socket
- ❑ TCP/IP is a **stream-oriented protocol** (data flows in a stream)
- ❑ Important Java libraries:
 1. Networking (e.g., java.net defines objects for sockets, accessing resources using HTTP)
 2. Data streams over a socket (java.io)
 3. Exception handling (e.g., java.lang)

Network programming

- ❑ Java supports the **host/port** IP addressing protocol
 1. The **InetAddress** class represents the IP address (host name and numeric name)
 2. The method:
 - `getHostName()` returns the host name
 - `getAddress()` returns the numeric name
- ❑ You can use the **InetAddress** to specify the host and port
- ❑ There are two types of socket objects
 1. The server side: **ServerSocket**
 2. The client side: **Socket**
- ❑ A process (i.e., program) on the:
 1. Server (remote host) listens on a port for requests from the client
 2. The client sends requests to the server using the same port

Network programming

- ❑ TCP offers two way communication using I/O streams



- ❑ Each socket has both `InputStream` and `OutputStream`
 1. The server's input stream is connected to the client's output stream
 2. The client's input stream is connected to the server's output stream
- ❑ Java provides methods to access the `InputStream` and `OutputStream`:
 1. `getInputStream()` to read from the socket
 2. `getOutputStream()` to write to the socket

Network programming: **Example**

- ❑ Example 1: Write a program that allows
 1. A client to send a message to a server
 2. The server then displays the message on the screen
- ❑ General steps
 1. Create a server process (i.e., executable program)
 2. Create a client process
 3. Run the server
 4. Run the client
- ❑ NOTE: always **close/stop the application before you run again** otherwise, an exception occurs
- ❑ Important
 1. The server and client programs are **two separate processes**
 2. You'll need to **catch exceptions** (the **stream, read/write, etc.**, can go wrong)
 3. You'll need to close all connections (**streams and sockets**)

Network programming: **Example**

- ❑ Note: import the appropriate Java APIs

```
//import the following
import java.net.*;
import java.io.*;
```

- ❑ Step 1: Create the server (**Server writes address to stream**) **Server.java**

```
//Server side: setup socket and stream, and write to the stream
ServerSocket sc=new ServerSocket(1888);//create a server socket object
Socket s=sc.accept();//server socket listens
OutputStream os=s.getOutputStream();//get input stream of the socket
DataOutputStream dos=new DataOutputStream(os);//combine streams
dos.writeUTF("Server: "+s.getLocalAddress());//write to stream
sc.close();// close the socket
```

- ❑ Step 2: Create the client (**Client reads from stream and writes to screen**) **Client.java**

```
//Client side: setup socket and stream, and write to the stream
Socket c=new Socket("localhost", 1888);//create socket
InputStream in=c.getInputStream();//get the server's output stream to read
DataInputStream dis =new DataInputStream(in);//combine to create output stream
String addr=dis.readUTF();
System.out.println(addr);
c.close();
```

Network programming: **Example**

❑ Don't forget:

1. Separate processes (i.e., the server and client applications)
2. Exception handling
3. Closing streams and sockets
4. Running applications: server process first, then client

Network programming: **Exercise**

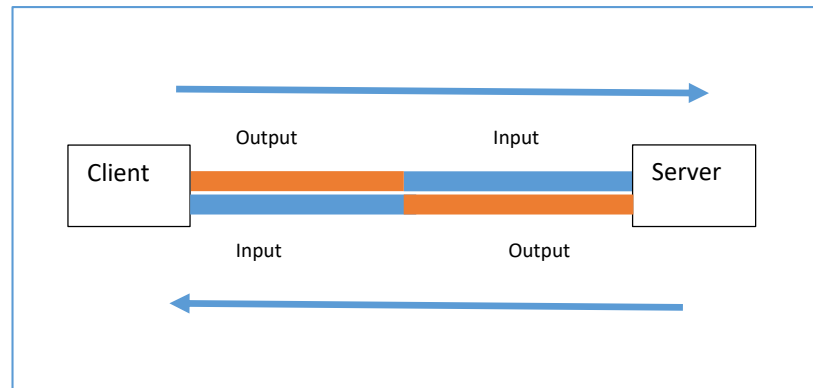
- ❑ Exercise: write a client/server application as ff:
 1. The client sends two numbers to the server
 2. The server returns the sum of the number to the client
 3. The client displays the sum on the screen

Network programming: **accessing URL resource on a network**

- ❑ URL (Universal Resource Locator) refers to a resource on a network
- ❑ URL has two parts: **protocol** and **resource name**
 - <https://www.oracle.com> (protocol: <http>; resource name: www.oracle.com)
- ❑ Java provides a class to create URL objects to connect to the host
 1. `URL url1 = new URL("www.oracle.com");`
 2. `URL url2 = new URL("http://localhost:8080/xmlweb/item.xml");`
 - Note: there are many different forms of the **URL** constructor
- ❑ You can also use URI (Universal Resource Identifier) class for URLs that contain special characters (e.g., "%", ",", etc.)
- ❑ Steps
 - a. Create URI object
 - b. Convert the URI to URL
- ❑ Example
 1. `URI uri=new URI("http://localhost:8080/my web/item.xml");`
 2. `URL url =uri.toURL();`
 - Note: **this approach is useful (to avoid any problems)**
- ❑ **Important**
 1. Streams (read and write)
 2. Exception handling

Network programming: **accessing URL resource on a network**

- ❑ Once connected to the host, documents can be accessed and downloaded in two ways:
 1. The easiest way is to use `openStream()` to **directly** read data
 2. Create `URLConnection` object with `openConnection()` and use
 - a. `getInputStream()` to read
 - b. `getOutputStream()` to write
 - c. Similar to the discussion on sockets



- ❑ Examples:
 1. Write a program to **read directly from** a given URL
 2. Write a program to read from a `URLConnection`
- ❑ See **Network programming : URL examples** for code

Comments and feedback

?

Next.....

Intersystem communication:
Part 3 (Remote Method
Invocation)